CYBER SECURITY PROJECT · BATCH-05

File Integrity Checker

A SHA-256 based File Integrity Monitoring (FIM) tool

Full Name	Faiq Umer
ID	ZYNVEX-CERT-2217
Project Title	File Integrity Checker
Language	Python 3 (standard library only)
Submission	November 2026

Table of Contents

1. Project Overview and Objectives
2. Security Concepts
3. Features and Technologies Used
4. Installation and Setup
5. Usage and Screenshots
6. System Workflow and Architecture
7. Testing and Results
8. Challenges and Limitations
9. Future Improvements
10. Conclusion
11. References

1. Project Overview and Objectives

The **File Integrity Checker** is a command-line tool that detects unauthorised changes to files. It first records a cryptographic fingerprint (SHA-256 hash) of every file in a trusted folder, called the *baseline*. On later runs it recomputes the fingerprints and compares them with the baseline, reporting every file that was **modified**, **newly added** or **deleted**.

This technique is called **File Integrity Monitoring (FIM)**. It is used by commercial and open-source security products to detect tampering such as altered configuration files, planted web shells, modified system binaries and deleted logs.

Objectives

- Understand and apply cryptographic hashing for integrity verification.
- Build a working tool that detects modified, added and deleted files.
- Produce clear, scriptable output (readable report and exit codes).
- Verify correctness with automated unit tests.
- Document the design, limitations and possible improvements.

2. Security Concepts

2.1 Integrity (the "I" in the CIA triad)

Integrity means data has not been altered in an unauthorised way. Checking integrity of files is a basic defensive control and appears in standards such as NIST SP 800-53 (SI-7) and PCI DSS (Requirement 11.5.2, change-detection mechanism).

2.2 Cryptographic hash functions

A hash function turns any input into a fixed-size digest. SHA-256 (defined in FIPS 180-4) produces a 256-bit value with these useful properties: the same input always gives the same digest; changing even one bit gives a completely different digest; and it is computationally infeasible to craft a different file with the same digest. A changed digest therefore proves the file content changed.

2.3 Baseline comparison

The baseline is a snapshot of the known-good state. Detection is a set comparison between the baseline and the current scan:

Condition	Meaning	Possible attack indicator
Path in both, hash differs	Modified	Tampered config, trojaned binary
Path only in current scan	New	Web shell, malware dropped
Path only in baseline	Deleted	Log wiping, removed evidence

3. Features and Technologies Used

Features

- SHA-256 hashing, read in 8 KB chunks so large files use very little memory.
- Recursive scan of all sub-folders; paths stored relative to the monitored folder.
- Reports modified, new and deleted files in a clear, colour-friendly format.
- Baseline stored as human-readable JSON; custom location via `--baseline`.
- Baseline file inside the monitored folder is skipped automatically.
- Exit codes: `0` intact / baseline created, `1` changes detected, `2` error. Useful for scripts and scheduled jobs.
- 7 automated unit tests.

Technologies

Item	Purpose
Python 3.8+	Implementation language
`hashlib`	SHA-256 hashing
`os`	Recursive folder traversal (`os.walk`)
`json`	Baseline storage
`argparse`	Command-line interface
`unittest`	Automated testing
Git / GitHub	Version control and source hosting

4. Installation and Setup

No third-party packages are required.

```
git clone https://github.com/<your-username>/file-integrity-checker.git
cd file-integrity-checker
python file_integrity_checker.py --help
```

Requirements: Python 3.8 or newer on Windows, Linux or macOS. Check with `python --version`.

5. Usage and Screenshots

Step 1 - Create the baseline

Run `init` on a folder you trust. The tool stores one SHA-256 hash per file.

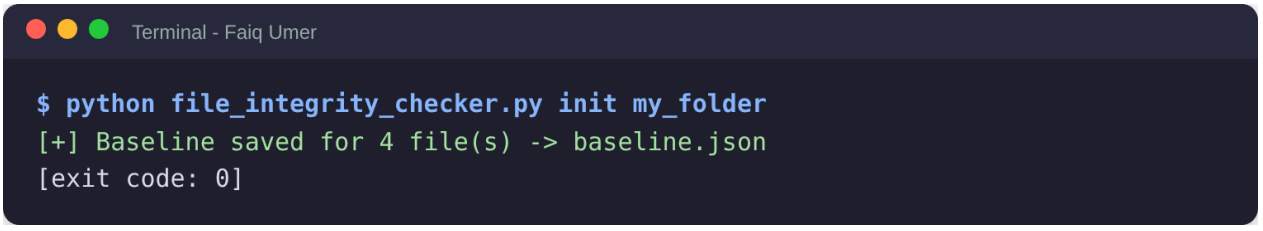


Figure 1: Baseline created for 4 files



Figure 2: Contents of baseline.json (relative path to SHA-256 hash)

Step 2 - Check an unchanged folder

```
Terminal - Faiq Umer

$ python file_integrity_checker.py check my_folder

===== Integrity Report =====
Modified : 0
New      : 0
Deleted  : 0

[OK] All files are intact. No changes detected.
[exit code: 0]
```

Figure 3: No changes detected, exit code 0

Step 3 - Simulate an attack, then check again

Three changes were made: `config.ini` was edited, `server.log` was deleted and a suspicious `shell.php` was added.

```
echo "admin:hacked" > my_folder/config.ini
rm my_folder/server.log
echo "backdoor" > my_folder/shell.php
```

```
Terminal - Faiq Umer

$ python file_integrity_checker.py check my_folder

===== Integrity Report =====
Modified : 1
  [~] config.ini
New      : 1
  [+] shell.php
Deleted  : 1
  [-] server.log

[WARNING] Changes detected!
[exit code: 1]
```

Figure 4: All three changes detected, exit code 1

Error handling

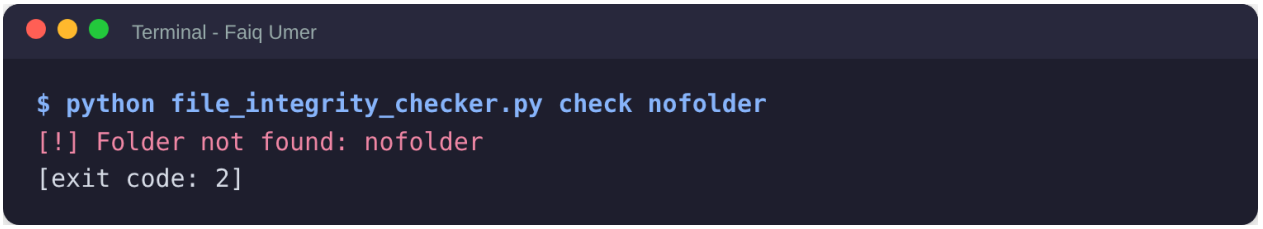


Figure 5: Missing folder gives a clear message and exit code 2

6. System Workflow and Architecture

The tool has four small components: a command-line interface, a folder scanner, a hashing function and a comparison/reporting stage. The baseline file connects the two modes.

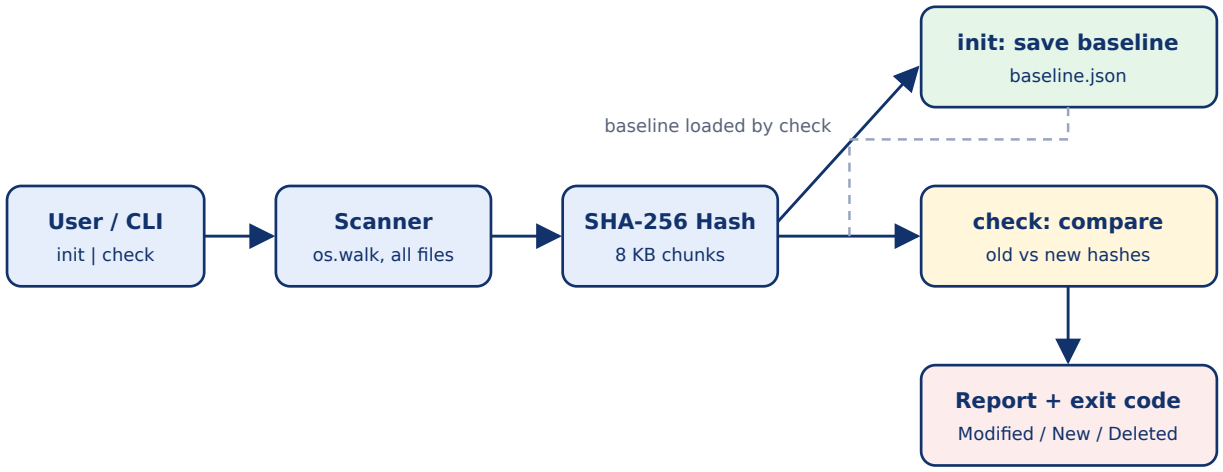


Figure 6: Workflow of the File Integrity Checker

Flow description

1. The user runs `init` or `check` with a folder path.
2. The scanner walks the folder recursively and opens each file in binary mode.
3. Each file is hashed with SHA-256 in 8 KB chunks.
4. **init:** the dictionary `{relative_path: hash}` is written to `baseline.json`.
5. **check:** the saved baseline is loaded and compared with the new scan.
6. The report lists modified, new and deleted files. The exit code is `0` (no change) or `1` (changes).

Key functions

Function	Role
`hash_file(path)`	Returns the SHA-256 hex digest of one file
`scan_folder(folder, skip)`	Returns `{relative_path: hash}` for the whole folder
`create_baseline(folder, file)`	Saves the scan as JSON
`compare(old, new)`	Returns sorted modified, added and deleted lists
`check_integrity(folder, file)`	Loads baseline, compares, prints report
`main(argv)`	Parses arguments and returns the exit code

7. Testing and Results

Seven unit tests were written with Python's `unittest`. Each test builds a temporary folder, so tests never touch real data. All tests pass.

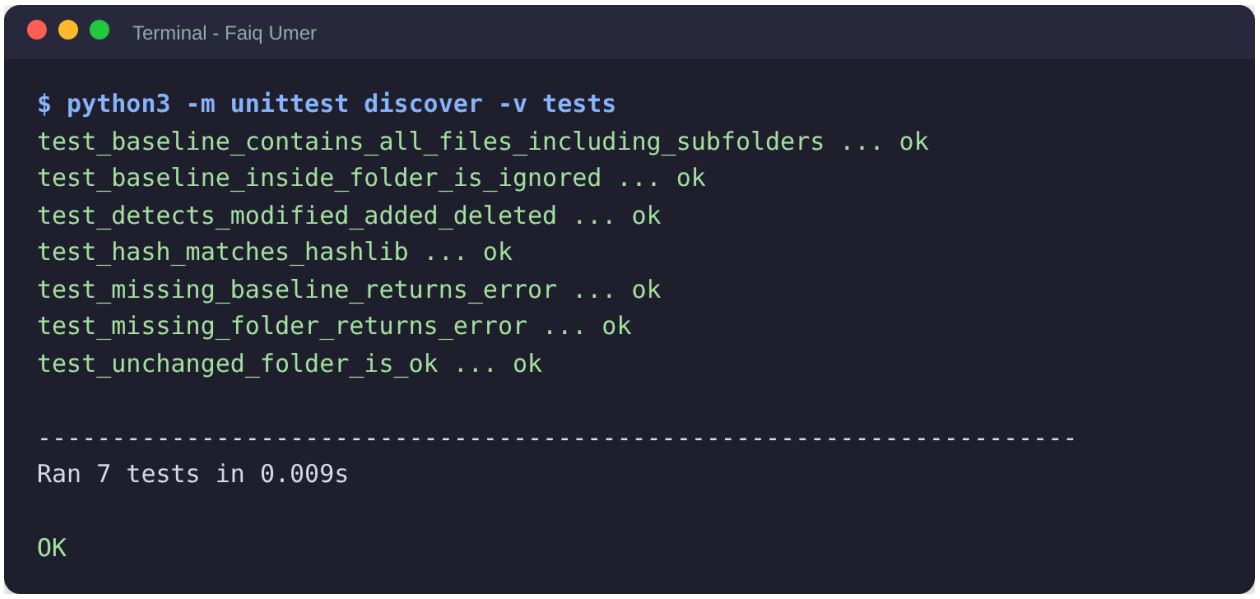


Figure 7: Unit test results (7 tests, all OK)

#	Test	Expected result	Result
1	Hash equals `hashlib` reference for known content	Digests match	Pass
2	Baseline includes files in sub-folders	All 3 files listed	Pass
3	Unchanged folder is checked	Exit code 0	Pass
4	One file edited, one deleted, one added	1 modified, 1 new, 1 deleted; exit code 1	Pass
5	Check without a baseline	Error, exit code 2	Pass
6	Folder does not exist	Error, exit code 2	Pass
7	Baseline stored inside monitored folder	Ignored; exit code 0	Pass

Manual test summary

In the demonstration run (Section 5), the tool correctly reported the edited `config.ini` as modified, the injected `shell.php` as new and the removed `server.log` as deleted, with no false positives for the untouched files `report.txt` and `docs/readme.md`.

8. Challenges and Limitations

Challenges

- **Large files:** reading a whole file into memory is wasteful, so files are hashed in small chunks.
- **Baseline inside the folder:** the baseline would otherwise show up as a "new" file. It is now skipped explicitly.
- **Sub-folders:** using relative paths keeps the baseline valid even if the folder is moved.
- **Testing safely:** temporary directories keep tests isolated and repeatable.

Limitations

- An attacker who can edit `baseline.json` can hide changes. The baseline should be kept on read-only or separate storage.
- Only file *content* is checked, not permissions, owner or timestamps.
- Checks run on demand; this is not real-time monitoring.
- Files that change legitimately (logs) will appear as modified.

9. Future Improvements

- Real-time monitoring (for example with the `watchdog` library).
- Protect the baseline with an HMAC or digital signature to detect tampering.
- Track permissions, ownership and modification time.
- Exclude patterns (for example `*.log`) and an "update baseline" command.
- Email or Slack alerts and scheduled runs (cron / Task Scheduler).
- A simple GUI or web dashboard.

10. Conclusion

The File Integrity Checker shows how a basic cryptographic primitive, SHA-256, can provide a practical security control. The tool reliably detects modified, added and deleted files, reports them clearly and reports its result through exit codes, so it can be used in scripts. It is covered by automated tests and uses only the Python standard library. Although it is a compact project, it demonstrates the same core idea as professional FIM products, and the limitations and future improvements above show how it could grow into a stronger monitoring solution.

11. References

1. NIST. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. 2015.
2. NIST. *SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*, control SI-7 (Software, Firmware, and Information Integrity). 2020.
3. PCI Security Standards Council. *PCI DSS v4.0*, Requirement 11.5.2 (change-detection mechanism).
4. Python Software Foundation. *hashlib — Secure hashes and message digests*. docs.python.org/3/library/hashlib.html
5. Python Software Foundation. *unittest — Unit testing framework*. docs.python.org/3/library/unittest.html
6. Python Software Foundation. *argparse — Parser for command-line options*. docs.python.org/3/library/argparse.html

Source code: *https://github.com/<your-username>/file-integrity-checker*